

closure_evidence_20260925

BOB-114 closure evidence — rate-limit detector self-validation golden-bad fixture

Revision: 1 **Last modified:** 2026-09-25T09:45:00Z **Item:** BOB-114 — BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector

Headline finding (§11.4.6 — stated as fact, not assumed)

No code change was made or was needed. docs/Issues.md lists BOB-114 as Status: Queued, but investigation of the live repository (git blame, git log, and a sibling doc's own closure note) proves the acceptance criteria this item describes were already fully implemented, committed, and mutation-tested **five weeks before this session started**:

- Commit 7b45113c6b03524d9c799bdd68496ec202575e4c (fix(002,T041): close the review's 5 IMPORTANT + 7 MINOR findings, and three defects nobody had reported, authored 2026-08-21) added the entire “Detector 2: rate limiting” self-validation block to challenges/scripts/ddos_resilience_challenge.sh — the rl_enforcing_server.py golden-good fixture, the rl_absent_server.py golden-bad fixture (never 429, never 5xx, no matter the burst — exactly the stub BOB-114 asks for), the rl_carrier_server.py golden-FALSE-with-carrier fixture, the rl_check() harness asserting the full (class × RED_MODE) verdict truth table, and the wiring of run_self_validation to run **automatically on every live invocation** (not just --self-validate).

- The same commit updated docs/testing/ddos_resilience.md line 571 from 4. ****Task**** (filed as ****BOB-114****): add a golden-bad synthetic fixture... to 4. ****Task**** (filed as ****BOB-114****, ****CLOSED 2026-08-21****): add a golden-bad synthetic fixture..., with a full write-up of the RED that proved the original gap (a \$1.1 mutation making the 429 tally always report 999 left the *old* self-test — the one with no rate-limit fixture at all — fully green) and three additional M1/M2/M3 mutations proving the new detector is load-bearing (see docs/testing/ddos_resilience.md lines 142–172, 257–263, 306–319 for the full historical write-up this evidence file does not duplicate).
- `git status --porcelain -- challenges/scripts/ddos_resilience_challenge.sh` is empty (clean, matches HEAD) both before and after this investigation; sha256sum of the file is identical before and after (c8fbc23e5a489e4047b1e77bb5fa81202f0f60696382a04551ac4b6bbf5e6df8).

Conclusion: docs/Issues.md’s Status: Queued for BOB-114 is a stale tracker entry — a doc/DB-sync gap (the code and a sibling doc both record **CLOSED 2026-08-21**, but the Issues.md tracker was never updated to match). Per my task’s explicit scope instructions I did **not** touch docs/workable_items.db or run the workable-items CLI to reconcile this — that reconciliation is left to the caller/operator. This file records what I found and the fresh independent verification I performed instead of fabricating a “before” state that does not exist in the live repository.

Files created/modified

- **Created:** docs/qa/BOB-114/closure_evidence_20260925.md (this file).
- **Modified:** none. challenges/scripts/ddos_resilience_challenge.sh was read and executed (unmodified) as part of verification; no edits were made to it or to any other tracked file. Two throwaway mutated *copies* were created and run **outside the repository**, under /tmp/claude-1000/.../scratchpad/bob114/, and never touched the tracked file (see “Independent mutation verification” below).

What the existing implementation does (the pattern, for verification)

The existing golden-bad fixture for the rate-limit detector mirrors the existing crash-resistance golden-bad fixture's exact pattern — same shape, same harness, same seam:

- **Crash-resistance golden-bad** (`bad_server.py`, pre-existing since BOB-074): a `ThreadingHTTPServer` bound to an ephemeral port (0, kernel picks the port, printed on stdout as `PORT=<n>` and read back — never a fixed port) that always answers 500.
`run_curl_status_probe` drives `SV_PROBE_N=12` requests at `SV_PROBE_C=6` against it; the assertion is `bad_5xx == bad_n` (every single response is a 500), sample-relative against `SV_MIN_SAMPLE=8`, never against a hardcoded literal count.
- **Rate-limit golden-bad** (`rl_absent_server.py`, added by BOB-114/commit 7b45113): the **same** `ThreadingHTTPServer-on-port-0` pattern, always answers 200 and **never** emits 429 or any 5xx, no matter how many requests land — i.e. exactly “a stub server that never returns HTTP 429 or 503 under burst load,” which is the item's own acceptance text, verbatim. It is driven through the **same** `run_curl_status_probe` harness, then fed to `ratelimit_classify()` (the real, single-copy detector function the live run also calls — never a duplicate) and asserted to classify as “absent”.

Both fixtures use the identical port-0-and-read-back start-up idiom (`sv_start_server`), the identical bounded-sample harness (`run_curl_status_probe` / `SV_PROBE_N` / `SV_PROBE_C` / `SV_MIN_SAMPLE`), and the identical real-process-real-HTTP-real-curl mechanism — **not a mock, not a description**: a genuine local Python process is forked, its kernel-assigned port is read back from its own stdout, and real `curl` requests hit real TCP sockets on `127.0.0.1`.

Three fixtures are shipped for detector 2, not just the required one:

Fixture	<code>rl*_server.py</code>	Behaviour	Must classify
golden-good	<code>rl_enforcing_server.py</code>	200 up to a configurable threshold (LIMIT), 429 + Retry-After: 1 past it	engaged

Fixture	rl*_server.py	Behaviour	Must classify
golden-bad (BOB-114's ask)	rl_absent_server.py	always 200; never 429, never any 5xx	absent
golden-FALSE-with-carrier	rl_carrier_server.py	always 200, but the headers/body <i>textually</i> <i>advertise</i> a rate limit it never enforces	absent (the false- positive guard)

Plus the full (class × RED_MODE) verdict-kind truth table is asserted (ratelimit_verdict_kind): engaged/GREEN→PASS, engaged/RED→FAIL, absent/GREEN→SKIP(extension_absent), absent/RED→PASS.

Before/after --self-validate output

“Before” — no RED baseline exists to capture, honestly stated

Per the anti-bluff mandate (§11.4.6/§11.4.1), I am not fabricating a “before this fixture existed” run against the live repository, because that state does not exist in this checkout — the fixture has been present and committed at every commit this session has touched. The genuine historical RED (captured 2026-08-21, before commit 7b45113 landed) is already recorded in docs/testing/ddos_resilience.md lines 142–150 and 306–319: “a §1.1 mutation making the 429 tally always report 999 (so the detector always claims ‘rate limiting engaged’) left the old self-test fully green” — i.e. before BOB-114, the self-test had no rate-limit fixture at all, so a completely decorative/blind detector that always says “engaged” would have passed every self-validation run undetected. That is the exact gap this item’s acceptance criteria describe, and it was already closed at that commit.

What I *did* capture fresh, in this session, against the current (unmodified) file:

Current state — bash challenges/scripts/ ddos_resilience_challenge.sh --self-validate

Run 2026-09-25T09:33Z, exit code 0:

```
=== ddos_resilience_challenge: self-validate ===
--- detector 1: crash resistance ---
  golden-good: n=12 5xx=0 conn_fail=0 (want 5xx=0 conn_fail=0)
  golden-bad:  n=12 5xx=12 conn_fail=0 (want 5xx=n)
--- detector 2: rate limiting ---
  fixture 'enforcing': n=12 429=10 5xx=0 -> class=engaged
(want=engaged) verdict GREEN=PASS RED=FAIL
  fixture 'absent': n=12 429=0 5xx=0 -> class=absent
(want=absent) verdict GREEN=SKIP RED=PASS
  fixture 'carrier': n=12 429=0 5xx=0 -> class=absent
(want=absent) verdict GREEN=SKIP RED=PASS
--- detector 3: sibling responsiveness ---
  fixture 'ok200': code=200 rc=0 retry_after='' ->
class=responsive (want=responsive)
  fixture 'r429_valid_seconds': code=429 rc=0 retry_after='30' ->
class=responsive (want=responsive)
  fixture 'r429_valid_httpdate': code=429 rc=0 retry_after='Wed,
21 Oct 2015 07:28:00 GMT' -> class=responsive (want=responsive)
  fixture 'r429_missing': code=429 rc=0 retry_after='' ->
class=degraded (want=degraded)
  fixture 'r429_garbage': code=429 rc=0 retry_after='soon-ish' ->
class=degraded (want=degraded)
  fixture 'r429_zero': code=429 rc=0 retry_after='0' ->
class=degraded (want=degraded)
  fixture 'r429_neg': code=429 rc=0 retry_after='-5' ->
class=degraded (want=degraded)
  fixture 'r429_baddate': code=429 rc=0 retry_after='Sun, 32 Nov
1994 25:99:99 GMT' -> class=degraded (want=degraded)
  fixture 'srv500': code=500 rc=0 retry_after='' -> class=degraded
(want=degraded)
  fixture 'refused': code=000 rc=7 retry_after='' ->
class=degraded (want=degraded)
  decision 'all-responsive': rc=0 (want 0; expected=3,
log_bytes=115)
  decision 'one-degraded': rc=1 (want 1; expected=5,
log_bytes=170)
  decision 'empty-log': rc=1 (want 1; expected=2, log_bytes=0)
  decision 'blank-line': rc=1 (want 1; expected=2, log_bytes=1)
  decision 'whitespace-only': rc=1 (want 1; expected=2,
log_bytes=6)
  decision 'torn-degraded': rc=1 (want 1; expected=1,
log_bytes=33)
  decision 'torn-mixed': rc=1 (want 1; expected=2, log_bytes=39)
  decision 'partial': rc=1 (want 1; expected=2, log_bytes=19)
  decision 'complete-2of2': rc=0 (want 0; expected=2,
log_bytes=53)
```

PASS: self-validate – crash-resistance, rate-limit AND sibling-responsiveness
detectors each distinguish their golden-good from their golden-bad
fixture; the rate-limit detector does NOT fire on the advertises-but-never-enforces carrier; and the sibling detector holds
the line against four 429s that DO carry a Retry-After header whose
value is not well-formed, plus both polarities of the isolation
decision and its empty-log fail-closed arm
(§11.4.107(10) + §11.4.201(1)(6)(7)(a))

EXIT_CODE=0

The fixture 'absent' line (class=absent (want=absent) verdict GREEN=SKIP RED=PASS) is BOB-114's specific acceptance criterion satisfied: the rate-limit detector, run against a stub server that never returns 429/503 under burst load, correctly reports the **absence** of rate limiting. The fixture 'enforcing' line is its golden-good counterpart (class=engaged) — already symmetric, confirming the detector still correctly recognizes a working limiter and was not broken by the golden-bad addition.

Independent mutation verification — proving the fixture is genuinely load-bearing

To produce **fresh, session-local** evidence (rather than relying solely on the historical M1/M2/M3 mutation table already recorded in docs/testing/ddos_resilience.md) that the golden-bad fixture is load-bearing and not decorative, I copied the unmodified script to two scratch files outside the repository (/tmp/claude-1000/-home-milosvasic-Projects-boba/ff28c9c0-dc2f-4e10-ab80-b979ad534575/scratchpad/bob114/) and applied one minimal, verified mutation to each — **the tracked repository file was never touched**; git status --porcelain was clean and the file's sha256 was identical before and after this entire session
(c8fbc23e5a489e4047b1e77bb5fa81202f0f60696382a04551ac4b6bbf5e6df8).

Mutation A — ratelimit_classify always returns "engaged"

This simulates exactly the decorative/blind-positive detector BOB-114 exists to catch: a detector that would silently certify a completely unprotected deployment as rate-limited.

```
ratelimit_classify() {
    local n429
    n429=$(tally_field "$1" '^429')
-   if [ "${n429:-0}" -gt 0 ] 2>/dev/null; then echo "engaged";
        else echo "absent"; fi
+   echo "engaged"
}
```

Result — `bash mut_always_engaged.sh --self-validate`, exit code 1:

```
--- detector 2: rate limiting ---
    fixture 'enforcing': n=12 429=10 5xx=0 -> class=engaged
(want=engaged) verdict GREEN=PASS RED=FAIL
    fixture 'absent': n=12 429=0 5xx=0 -> class=engaged
(want=absent) verdict GREEN=PASS RED=FAIL
    [SELF-VAL BAD] rate-limit detector classified 'absent' as
'engaged', expected 'absent'
    [SELF-VAL BAD] rate-limit verdict table wrong for 'absent':
GREEN=PASS (want SKIP) RED=FAIL (want PASS)
    fixture 'carrier': n=12 429=0 5xx=0 -> class=engaged
(want=absent) verdict GREEN=PASS RED=FAIL
    [SELF-VAL BAD] rate-limit detector classified 'carrier' as
'engaged', expected 'absent'
    [SELF-VAL BAD] rate-limit verdict table wrong for 'carrier':
GREEN=PASS (want SKIP) RED=FAIL (want PASS)
...
FAIL: self-validate - detector honesty check failed
```

The golden-bad (absent) fixture — the one BOB-114 asks for — is exactly what catches this mutation. **This is direct, fresh proof the fixture is load-bearing, not decorative:** a broken/blind detector that always claims rate limiting is engaged is caught immediately by `--self-validate`, and would (per the `run_self_validation` wiring landed alongside it) refuse to mint any live verdict at all (FAIL: `ddos_resilience_challenge - detector self-validation FAILED; refusing to mint live verdicts from unvalidated instrumentation ($11.4.115(F))`), the guard printed by the live-run branch a few lines below `run_self_validation` in the same file).

Mutation B — ratelimit_classify always returns "absent"

The symmetric check, confirming the golden-good counterpart (enforcing) is equally load-bearing — the detector must not have been broken in the “can it still see a real limiter” direction either.

```
ratelimit_classify() {
    local n429
    n429=$(tally_field "$1" '^429')
-   if [ "${n429:-0}" -gt 0 ] 2>/dev/null; then echo "engaged";
        else echo "absent"; fi
+   echo "absent"
}
```

Result — bash mut_always_absent.sh --self-validate, exit code 1:

```
--- detector 2: rate limiting ---
    fixture 'enforcing': n=12 429=10 5xx=0 -> class=absent
(want=engaged) verdict GREEN=SKIP RED=PASS
    [SELF-VAL BAD] rate-limit detector classified 'enforcing' as
'absent', expected 'engaged'
    [SELF-VAL BAD] rate-limit verdict table wrong for 'enforcing':
GREEN=SKIP (want PASS) RED=PASS (want FAIL)
    fixture 'absent': n=12 429=0 5xx=0 -> class=absent
(want=absent) verdict GREEN=SKIP RED=PASS
    fixture 'carrier': n=12 429=0 5xx=0 -> class=absent
(want=absent) verdict GREEN=SKIP RED=PASS
...
FAIL: self-validate – detector honesty check failed
```

This confirms: **(a)** the golden-bad fixture (absent) genuinely detects a broken “always says engaged” detector (Mutation A), **(b)** the golden-good fixture (enforcing) genuinely detects a broken “always says absent” detector (Mutation B) — the detector’s ability to correctly report rate limiting as **present** when it IS present was not broken and remains covered, satisfying the requested symmetry check — and **(c)** the unmutated, real, committed detector passes both directions cleanly (the “current state” run above).

Scratch-file integrity (control needle)

```
$ sha256sum mut_always_engaged.sh mut_always_absent.sh
bf524af602d67ba307cf9e5395d878ba6878771656a4a53950e8c95e3bff8504
mut_always_engaged.sh
3e095f995a0b76455762ae536c6ac6f7b87bb7dc36f6f7500beab843daf2f98c
mut_always_absent.sh
```

```
$ diff <original> mut_always_engaged.sh
212c212
```

```

<  if [ "${n429:-0}" -gt 0 ] 2>/dev/null; then echo "engaged";
else echo "absent"; fi
---
>  echo "engaged"

$ diff <original> mut_always_absent.sh
212c212
<  if [ "${n429:-0}" -gt 0 ] 2>/dev/null; then echo "engaged";
else echo "absent"; fi
---
>  echo "absent"

```

Both diffs are exactly and only the intended one-line change; both mutated files' sha256 differ from the original (c8fbc23e5a489e4047b1e77bb5fa81202f0f60696382a04551ac4b6bbf5e6df8), proving the mutation actually applied (§11.4.6/§11.4.201 control-needle discipline — the mutation harness itself must be verified to have fired before any conclusion is drawn from its result).

Assumptions the item text didn't fully specify

1. **“correctly reports the ABSENCE” was read as “classifies as absent AND drives the correct GREEN=SKIP/RED=PASS verdict kind”,** not merely as a bare string match — the existing implementation asserts both the classification and the full verdict-kind truth table for every fixture, which is a superset of what the item's acceptance text literally asks for.
2. **“never returns HTTP 429 or 503”** was read, and the existing `rl_absent_server.py` implements, as “never returns 429, and never returns *any* 5xx” (not narrowly 503 only) — the self-test's own assertion checks `n5xx == 0` for the non-enforcing fixtures, which is the stronger and more honest reading (a stub that returned 500 instead of 503 while still enforcing no rate limit would otherwise slip past a 503-only check).
3. Since the acceptance criteria were already met at HEAD, I read my task's “verification before/after” instruction as calling for **independent, fresh, in-session proof of load-bearing-ness** (the scratch-copy mutation testing above) rather than a literal repeat of a code-writing step that would have produced no diff. I did not invent a “before” state that never existed in the live repository, per the anti-bluff mandate that forbids fabricated RED baselines.
4. I did **not** attempt to reconcile `docs/Issues.md`'s stale `Queued` status, `docs/workable_items.db`, or run any `workable-items` CLI command, per the explicit scope restriction in my task instructions. That

reconciliation (updating the tracker to Status: Fixed (→ Fixed.md) or Completed, citing commit 7b45113 and the already-closed note in docs/testing/ddos_resilience.md:571) is left to the caller.